Neural Networks

- Lecture 2 -

MLP and Activation Functions

Outline

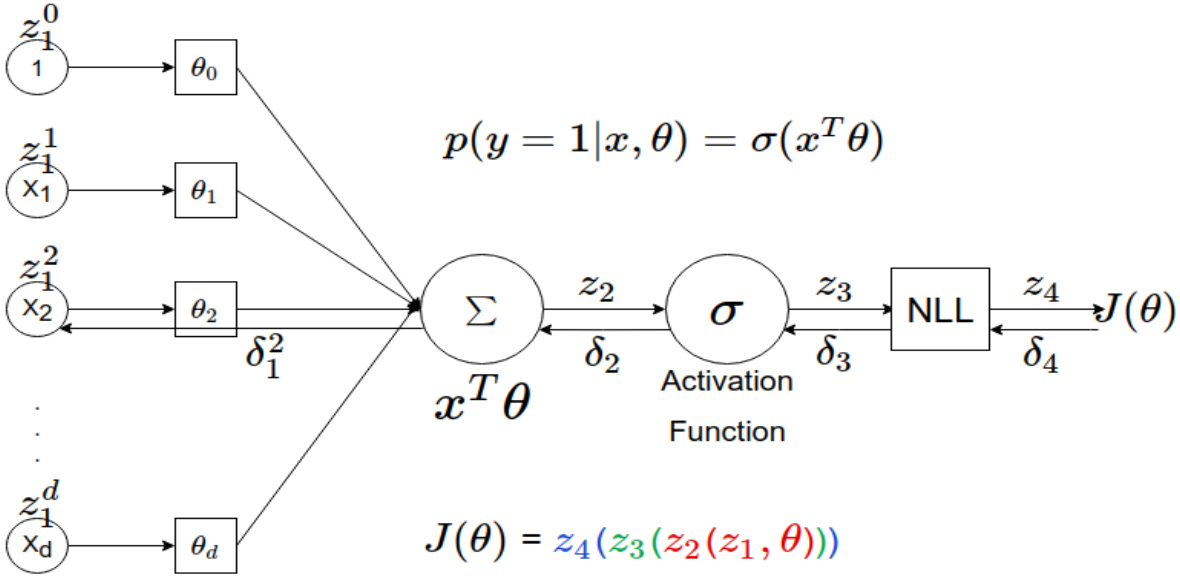
- Recaps
  - Computational Graph and Backpropagation
- Multi-Layered Perceptron
- Activation Functions

Recap

Backpropagation for a NN

Backpropagation for a “Logistic Regression NN”

Backpropagation for a “Logistic Regression Neural Net”



$$\delta_4 = J(\theta)$$

$$\delta_3 = \frac{\partial J}{\partial z_3} = \underbrace{\frac{\partial J}{\partial z_4}}_{1} \frac{\partial z_4}{\partial z_3}$$

$$\delta_3 = \frac{\partial -(y \log(z_3) + (1-y)\log(1-z_3))}{\partial z_3}$$

$$\delta_3 = \frac{-y}{z_3} - \frac{(1-y)}{(1-z_3)}$$

$$\delta_2 = \frac{\partial J}{\partial z_2} = \frac{\partial J}{\partial z_3}\frac{\partial z_3}{\partial z_2}$$

$$\delta_2 = \delta_3 \frac{\partial \sigma(z_2)}{\partial z_2} = \delta_3 \sigma(z_2)(1-\sigma(z_2))$$

$$\delta_1^2 = \frac{\partial J}{\partial z_1^2} = \delta_2 \frac{\partial \sum_{i=1}^{d} x_i \theta_i}{\partial x_2} = \delta_2 \theta_2$$

Backpropagation with Vectors

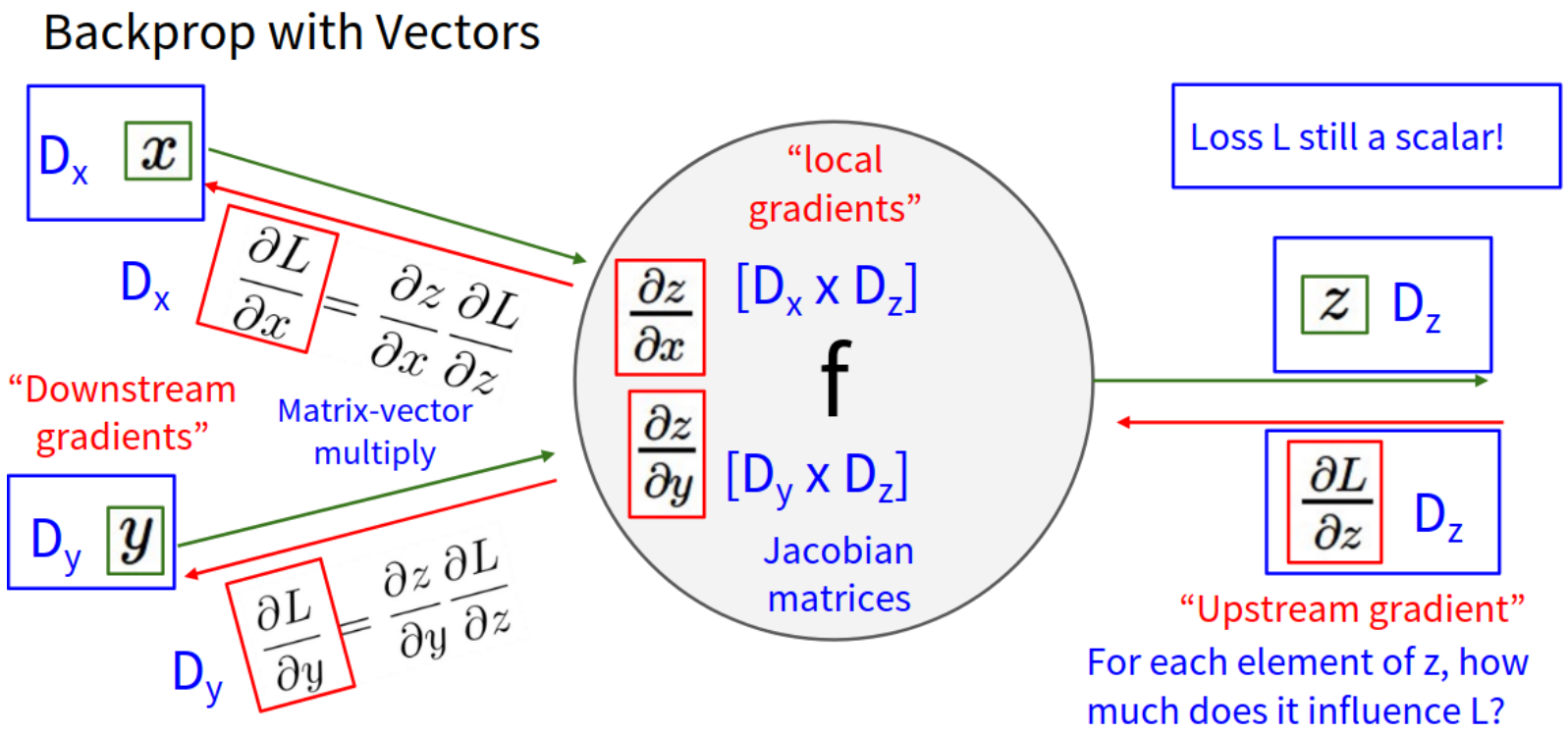


Diagram source: Fei-Fei Li, Ehsan Adeli, Stanford CS231 – Lecture 4

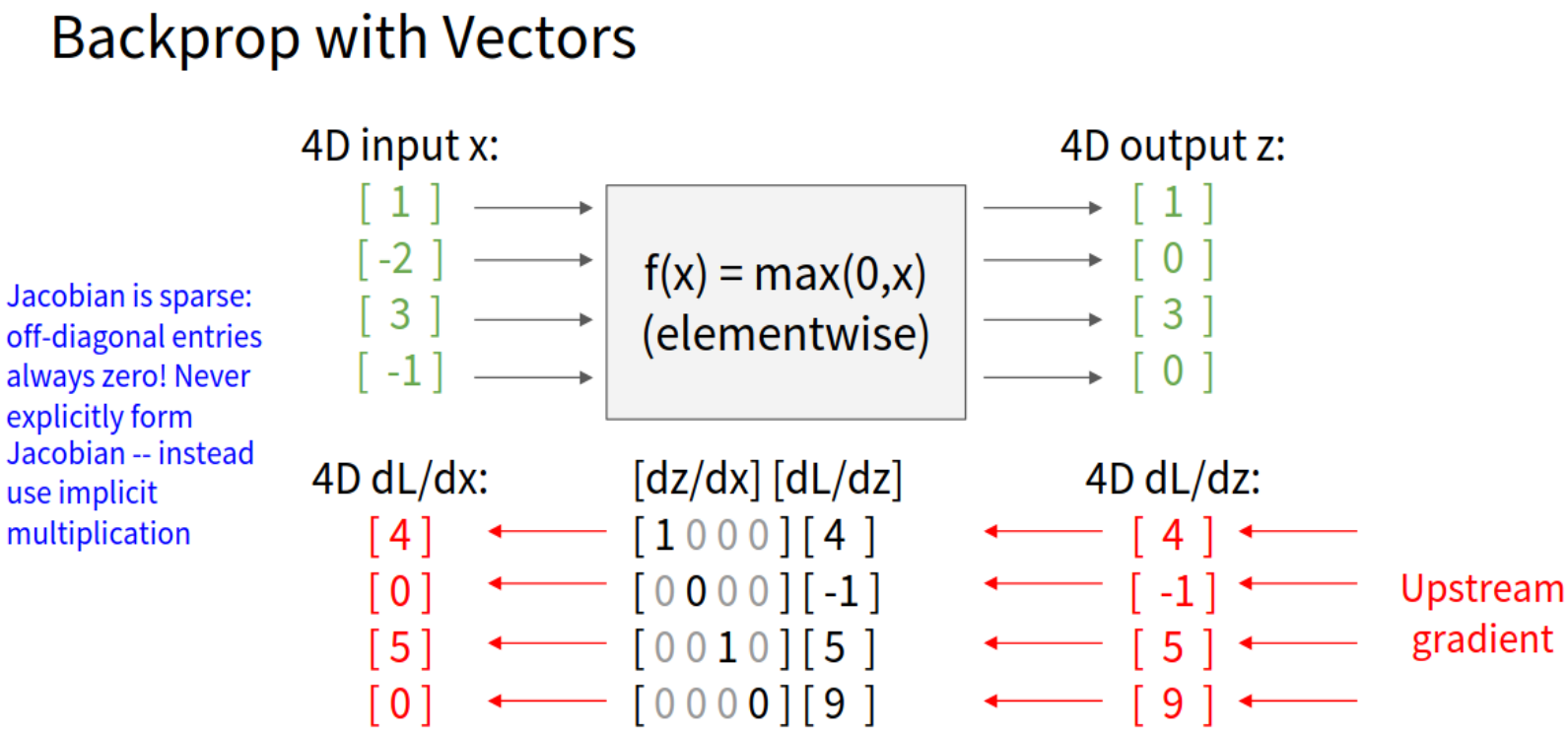


Diagram source: Fei-Fei Li, Ehsan Adeli, Stanford CS231 – Lecture 4

Backpropagation with Matrices

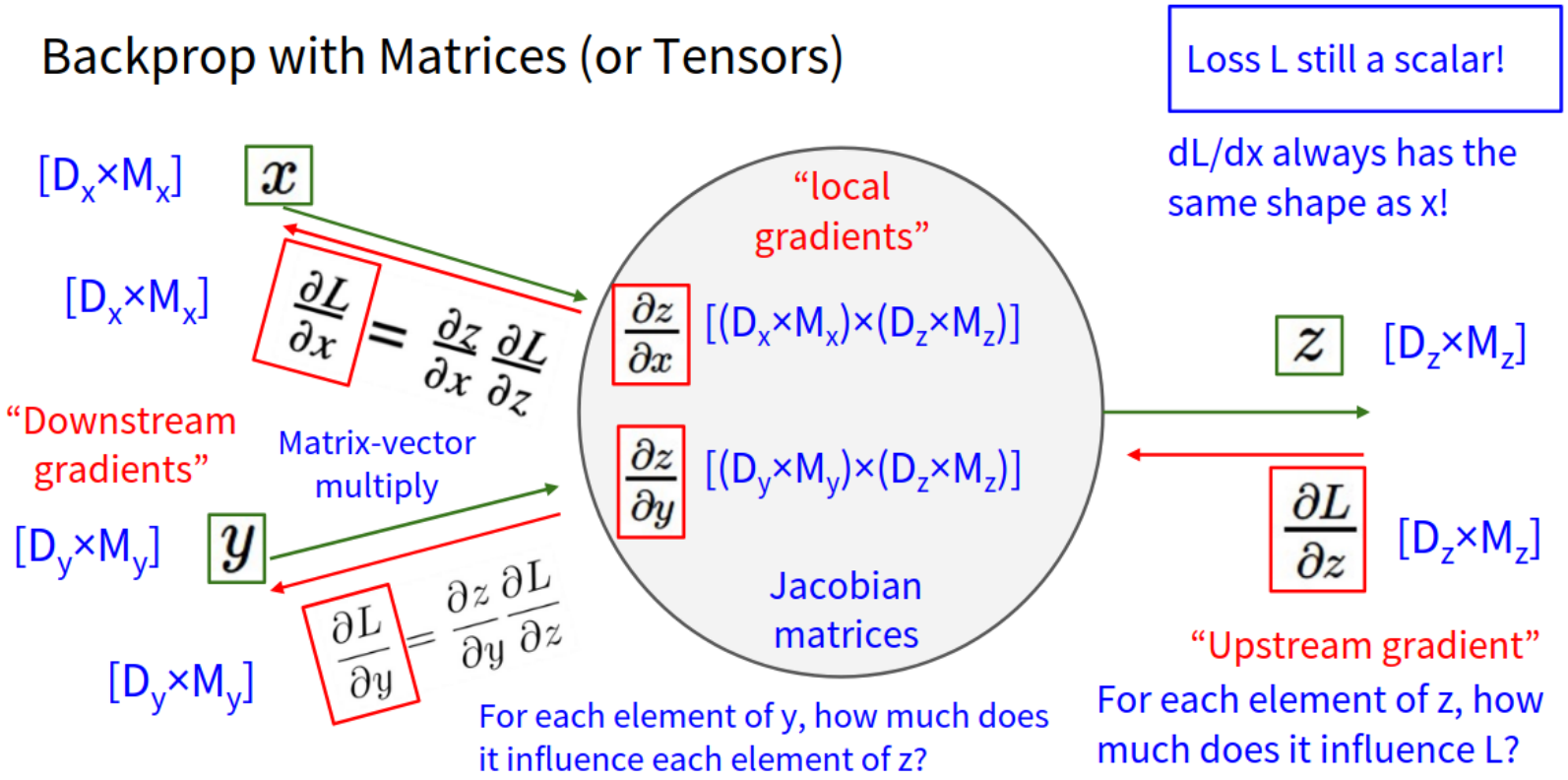


Diagram source: Fei-Fei Li, Ehsan Adeli, Stanford CS231 – Lecture 4

Backpropagation with Matrices

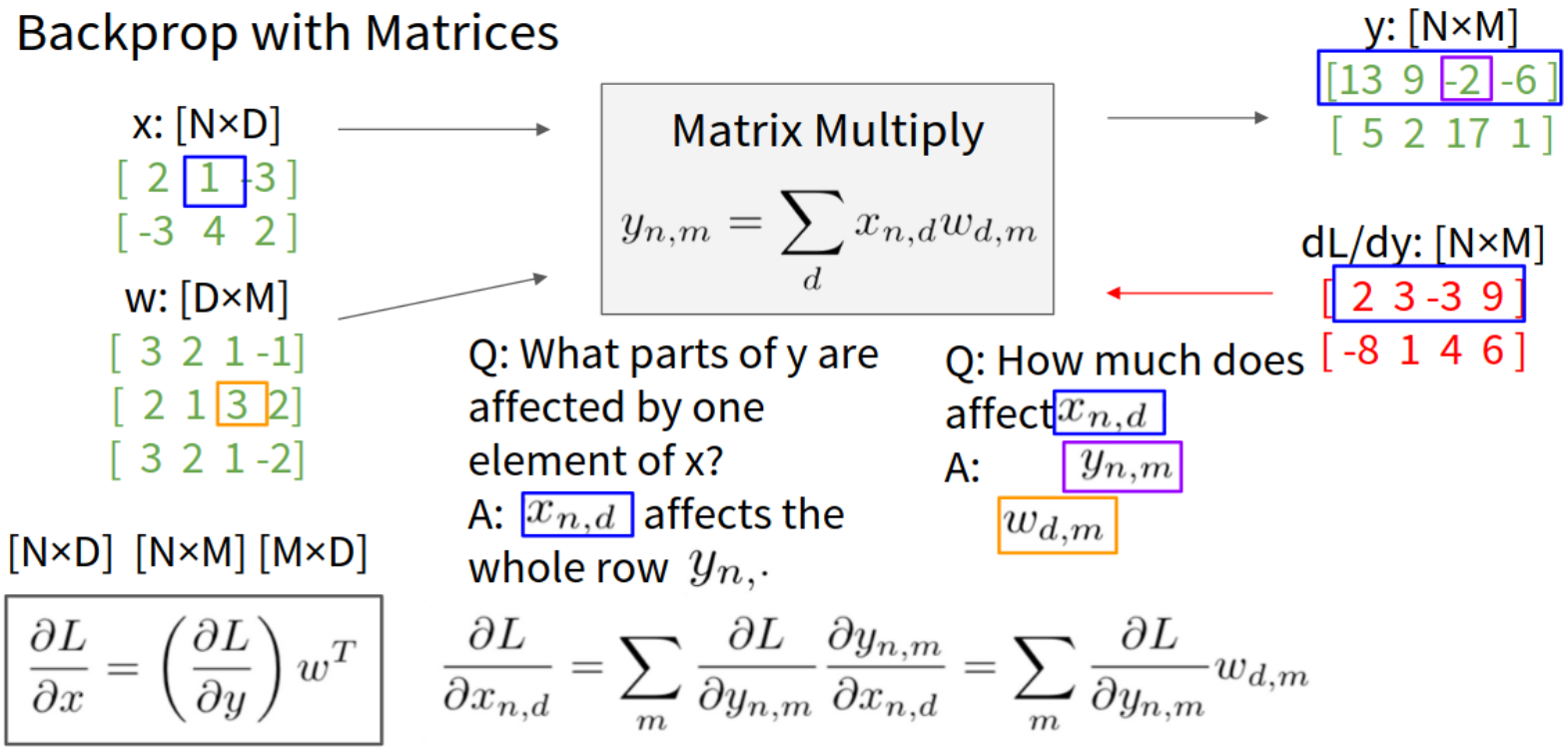
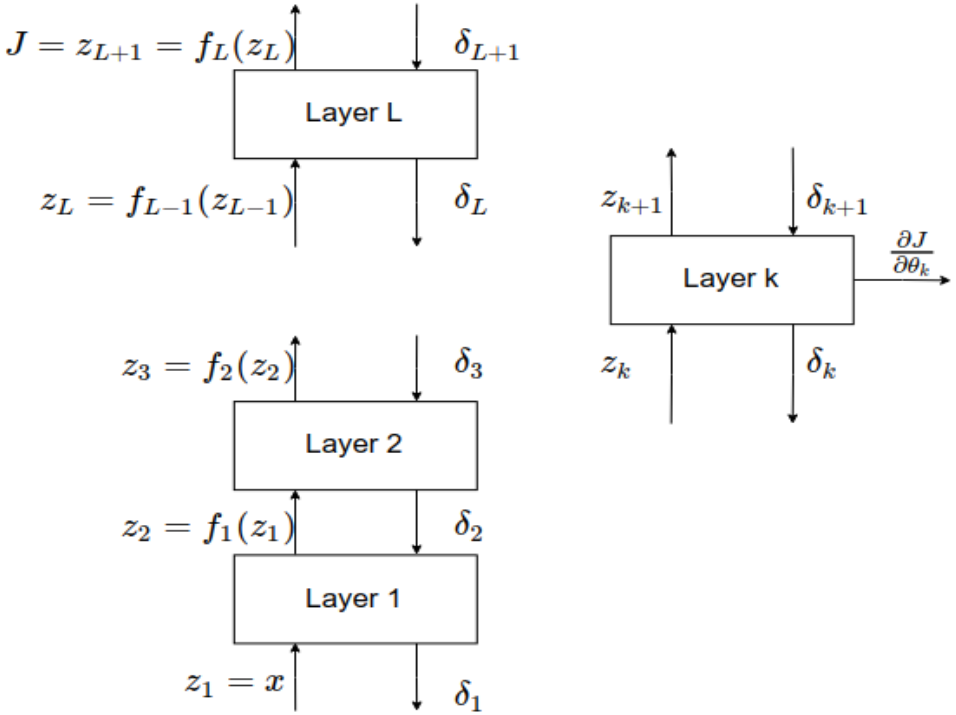


Diagram source: Fei-Fei Li, Ehsan Adeli, Stanford CS231 – Lecture 4

Backpropagation – Generalization to a Network Layer

In general



Forward pass:

$$z_{k+1} = f_k(z_k)$$

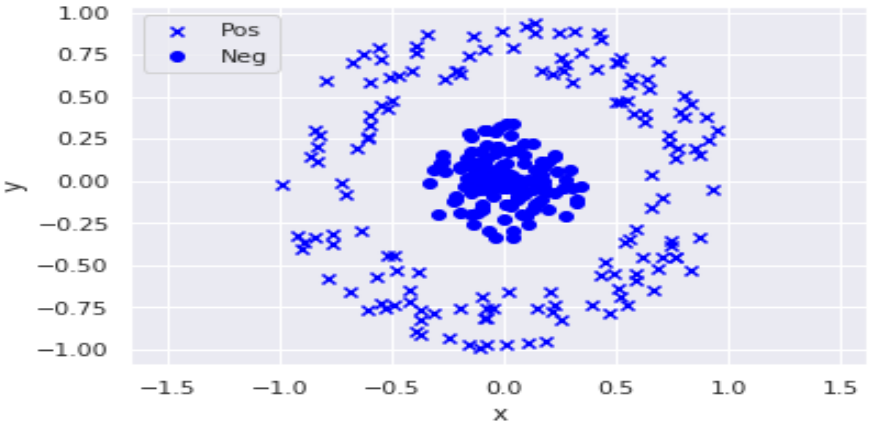
Backward pass:

$$\delta_k^i = \frac{\partial J}{\partial z_k^i} = \sum_j \frac{\partial J}{\partial z_{k+1}^j} \frac{\partial z_{k+1}^j}{\partial z_k^j} = \sum_j \delta_{k+1}^j \frac{\partial z_{k+1}^j}{\partial z_k^i}$$

$$\frac{\partial J}{\partial \theta_k} = \sum_j \frac{\partial J}{\partial z_{k+1}^j} \frac{\partial z_{k+1}^j}{\partial \theta_k} = \sum_j \delta_{k+1}^j \frac{\partial z_{k+1}^j}{\partial \theta_k}$$

Multi-Layered Perceptron

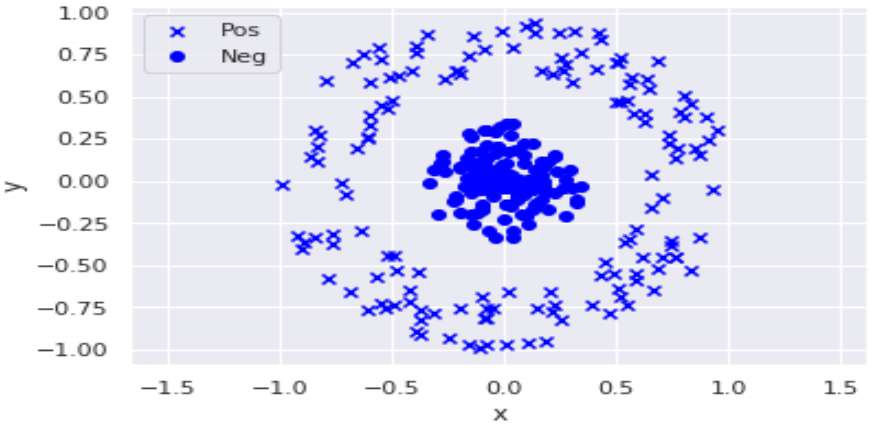
Consider the rings dataset



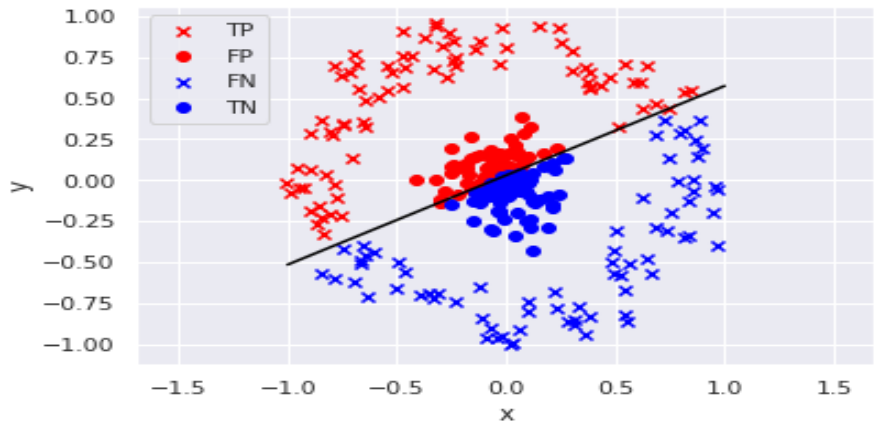
Rings dataset

What is the expected result of a Logistic Regression Classifier for this dataset?

Consider the rings dataset



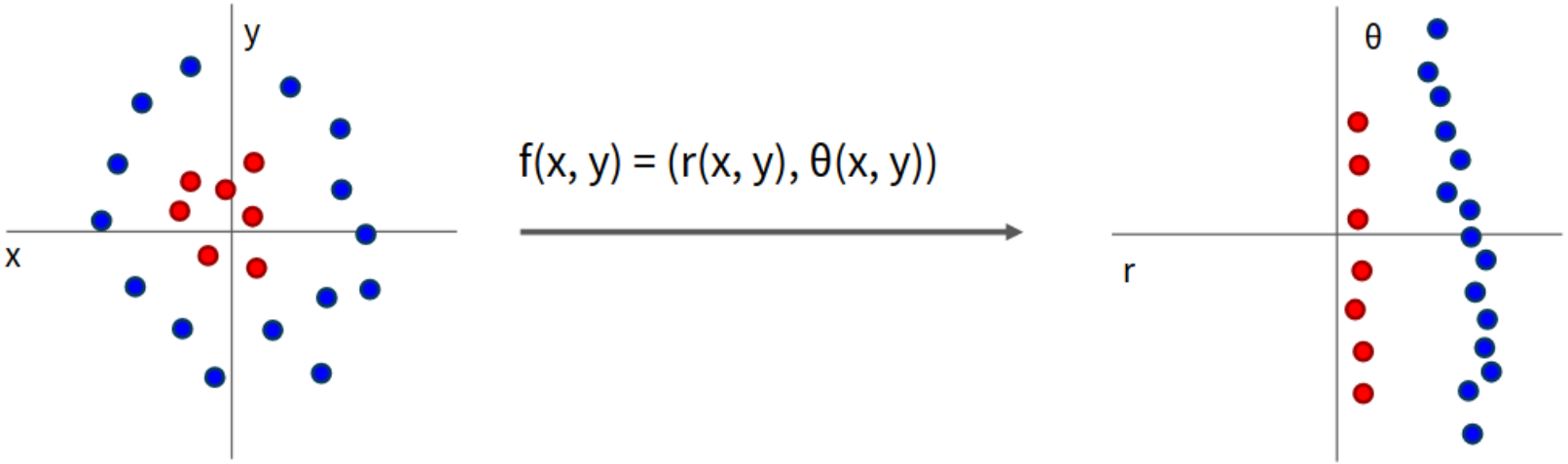
Rings dataset



Logistic regression result

Logistic regression is still insufficient for classification, because the dataset **is not linearly separable**

Consider the rings dataset. Why do we **want non-linearity?**



Cannot separate red and blue points with linear classifier

After applying feature transform, points can be separated by linear classifier

Diagram source: Fei-Fei Li, Ehsan Adeli, Stanford CS231 – Lecture 4

Multi-Layer Perceptron

- MLP – consists of multiple layers of nodes (neurons): an **input layer**, **one or more hidden layers**, and an **output layer**.
  - network is **fully connected** between consecutive layers
  - Use of a **nonlinear activation function** for each neuron

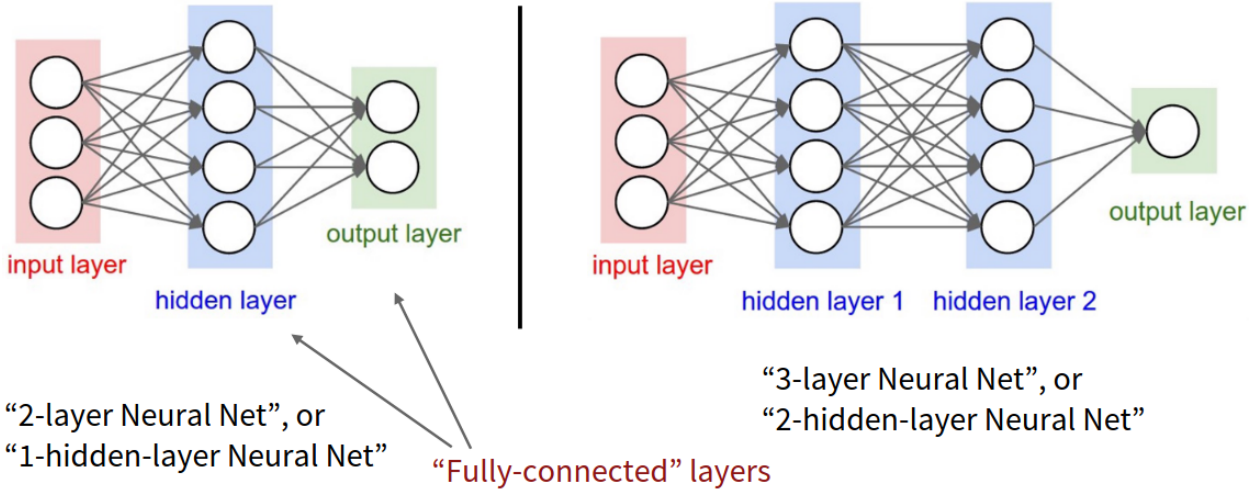
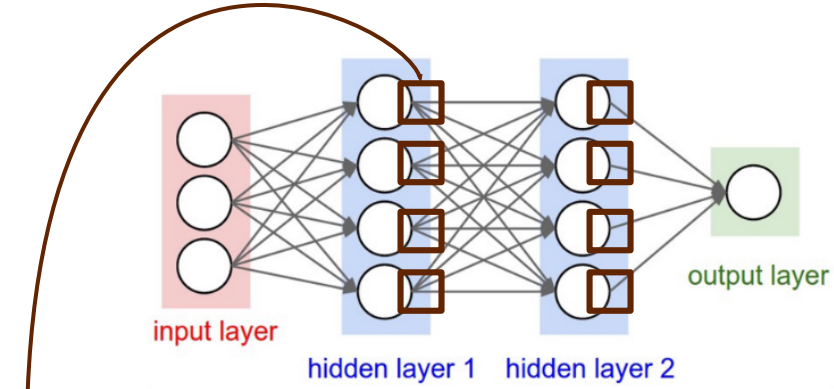


Diagram source: Fei-Fei Li, Ehsan Adeli, Stanford CS231 – Lecture 4

Multi-Layer Perceptron

- MLP – consists of multiple layers of nodes (neurons): an **input layer**, **one or more hidden layers**, and an **output layer**.
  - network is **fully connected** between consecutive layers
  - Use of a **nonlinear activation function** for the output of each neuron




```
# forward-pass of a 3-layer neural network:
f = lambda x: 1.0/(1.0 + np.exp(-x)) # activation function (use sigmoid)
x = np.random.randn(3, 1) # random input vector of three numbers (3x1)
h1 = f(np.dot(W1, x) + b1) # calculate first hidden layer activations (4x1)
h2 = f(np.dot(W2, h1) + b2) # calculate second hidden layer activations (4x1)
out = np.dot(W3, h2) + b3 # output neuron (1x1)
```

Diagram source: Fei-Fei Li, Ehsan Adeli, Stanford CS231 – Lecture 4

Multi-Layer Perceptron - Properties

- **Universal Function Approximator Theorem**: An MLP with **at least one hidden layer** and **a nonlinear activation function**, can *approximate any continuous function on a closed interval*, **given sufficient neurons**.
  - Neural Nets ***have the theoretical*** power to model complex patterns

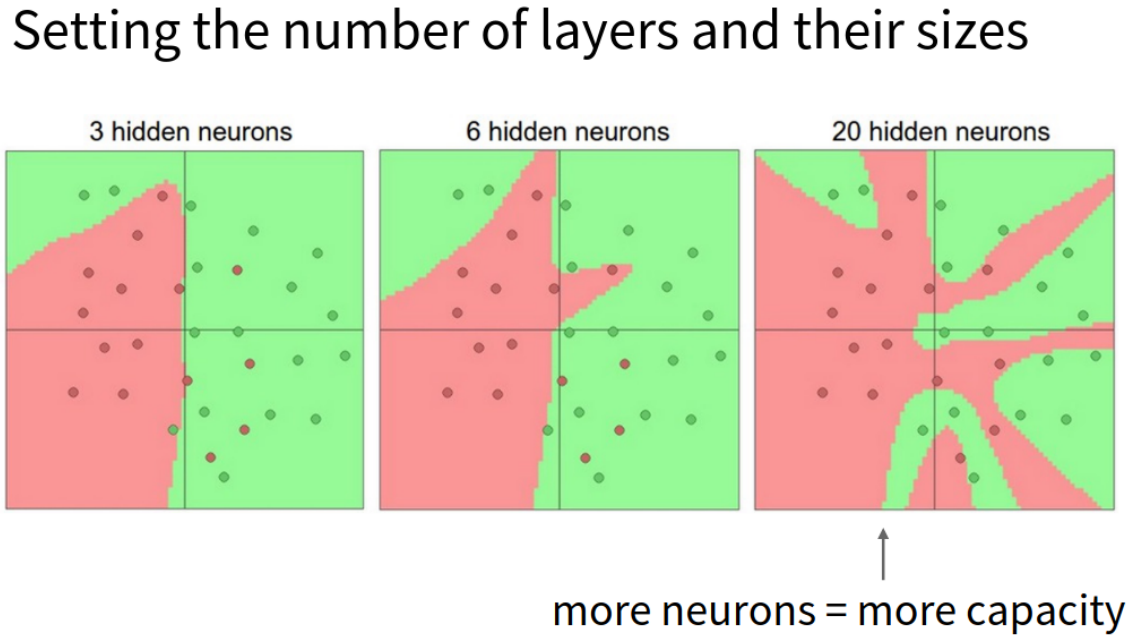
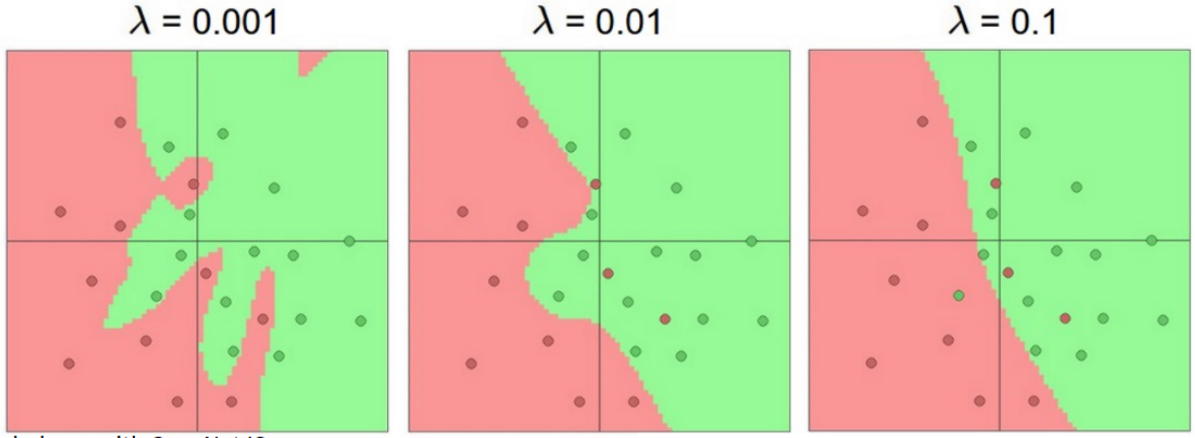


Diagram source: Fei-Fei Li, Ehsan Adeli, Stanford CS231 – Lecture 4

Multi-Layer Perceptron - Properties

- **Universal Function Approximator Theorem**: An MLP with **at least one hidden layer** and **a nonlinear activation function**, can *approximate any continuous function on a closed interval*, **given sufficient neurons**.
  - A rule of thumb in Deep Learning is that the **size** of a neural model **is not** used as a regularization. We want **expressive models** to which we add **strong regularization (if needed –** more in lecture on Normalization**)**



(Web demo with ConvNetJS: http://cs.stanford.edu/people/karpathy/convnetjs/demo/classify2d.html)

TensorFlow Play Ground: https://playground.tensorflow.org/

$$L(W) = \frac{1}{N}\sum_{i=1}^{N} L_i(f(x_i, W), y_i) + \lambda R(W)$$

Diagram source: Fei-Fei Li, Ehsan Adeli, Stanford CS231 – Lecture 4

Multi-Layer Perceptron - Properties

- Inductive Bias of MLPs: **Global Connectivity** - MLPs are fully connected
  - model **global dependencies** between inputs and outputs
  - does **not** inherently exploit local patterns (such as spatial or temporal locality) without explicit architectural changes

Activation Functions

Activation Functions

- Activation functions add **non-linear transformations** to the output of neurons, allowing neural networks to **model complex, real-world data**

Activation Functions

- Activation functions add **non-linear transformations** to the output of neurons, allowing neural networks to **model complex, real-world data**
  - Without activation functions, the network would only be able to represent linear mappings → expressivenes limited to that of a linear / logistic regression.

Activation Functions

- Activation functions add **non-linear transformations** to the output of neurons, allowing neural networks to **model complex, real-world data**
  - Without activation functions, the network would only be able to represent linear mappings → expressivenes limited to that of a linear / logistic regression.

- Enabling **Deep Learning**: applying non-linearity + stack multiple layers → **capture higher-level features** (e.g. edge in image, trend in time series, relations between words)

Activation Functions

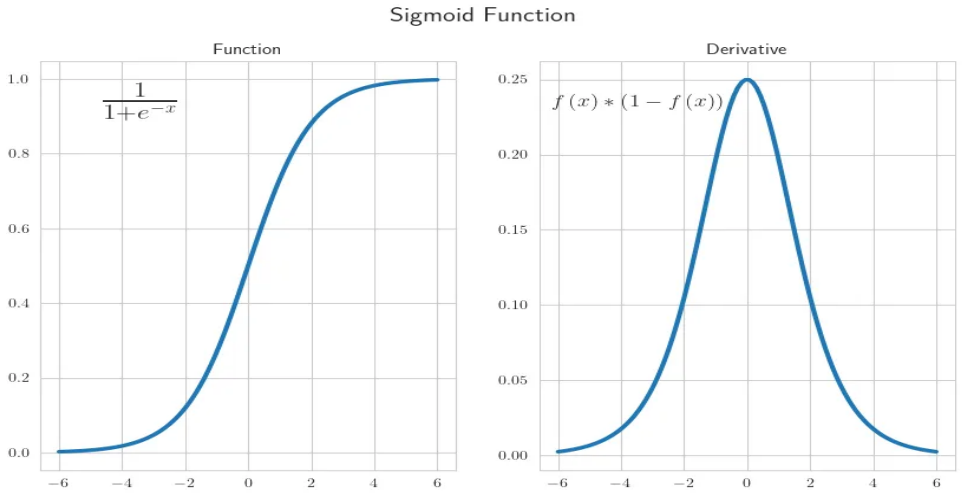
- Activation functions add **non-linear transformations** to the output of neurons, allowing neural networks to **model complex, real-world data**
  - Without activation functions, the network would only be able to represent linear mappings → expressivenes limited to that of a linear / logistic regression.

- Enabling **Deep Learning**: applying non-linearity + stack multiple layers → **capture higher-level features** (e.g. edge in image, trend in time series, relations between words)

- **Impact on Gradient Flow**: Non-linear functions affect the gradients computed during backpropagation
  - impact on weight updates during training
  - choice of activation function influences ease of learninng and convergence to an optimal solution

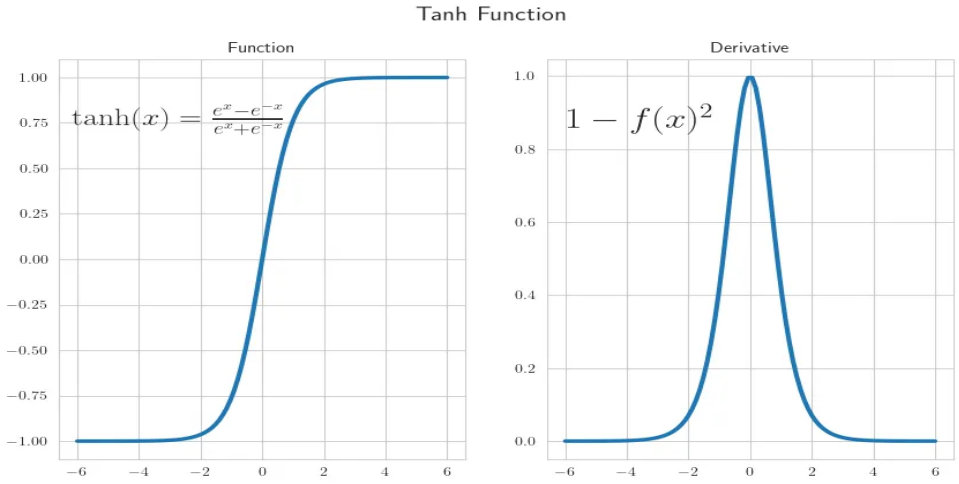
Activation Functions – Common Functions: Logistic Sigmoid

- Smooth step like function
- Range in [0, 1] → good for **final classifier** or as **a gating mechanism**
- Between X values -2 to 2, Y values are very steep
- Can “saturate” quickly:
  - towards either end of the sigmoid function, the Y values tend to respond much less to changes in X
  - Gradient in such regions is small → “vanishing gradient” → network may learn slowly or not at all



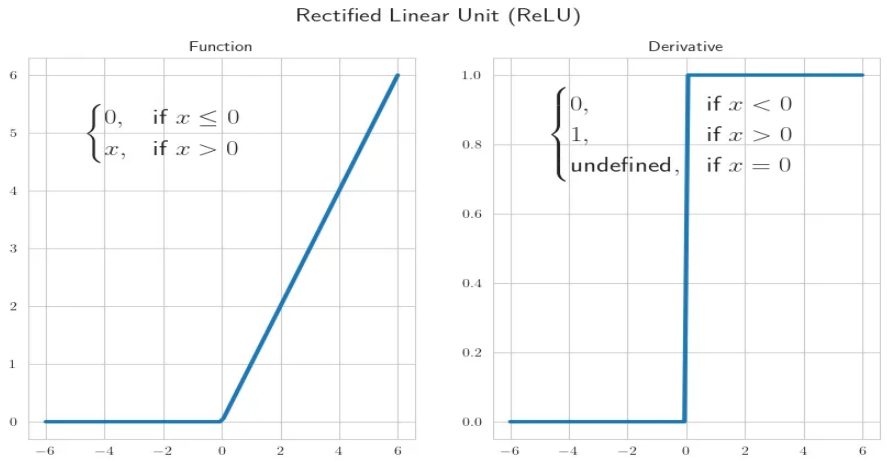
Activation Functions – Common Functions: tanh

- Similar in shape to sigmoid
- Range in [-1, 1]
- Output expectation is close to zero-mean → output values that are both positive and negative which can be an advantage for subsequent weights (weights themselves don't have to change value from positive to negative, if the input does) → faster convergence
- Stronger gradient than sigmoid (derivatives are steeper)
- Choice between sigmoid or tanh depends on requirement of problem (e.g. classification vs regression) and gradient strength
- Tanh requires more care about normalization



Activation Functions – Common Functions: ReLU

- Efficient computation
- Bound: [0, inf) → may cause large activations
  - Requires care in subsequent normalization
- Any function can be approximated with combinations of ReLU → we can stack layers → result in **piecewise linear decision boundaries**
- Dying ReLU problem
  - Neurons that get 0 gradient update may get into a region where they no longer activate => part of the network can become **passive** → no longer responsive to variations in input / error

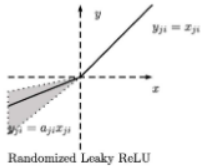


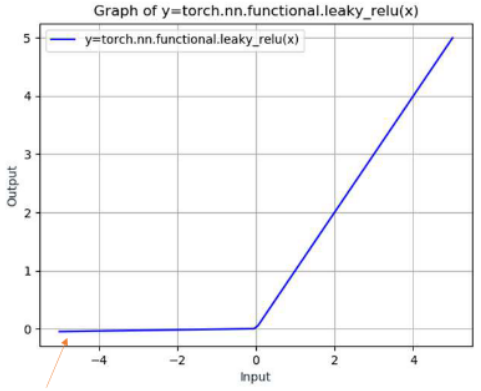
Activation Functions – Common Functions: Leaky ReLU

- Mitigate “dying ReLU”, by allowing negative gradients

Leaky ReLU

- $Output = \begin{cases} x & for\ x \geq 0 \\ e.g.\ 0.01 \cdot x & for\ x < 0 \end{cases}$
- Mitigates dying ReLu





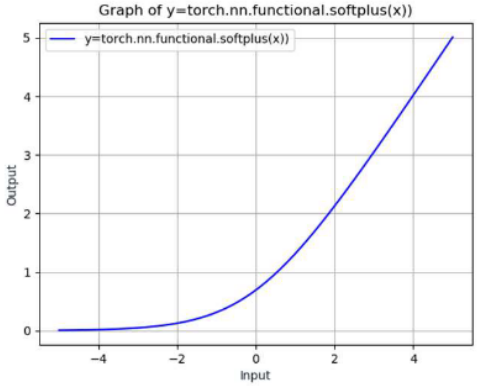
atcold.github.io

Activation Functions – Common Functions: Softplus

- Mitigate “dying ReLU”, by means of a continuously differentiable function
- Ensures smoother gradients than ReLU → more stable updates during backpropagation
- Smoother handling of outlier values (large negative values as input are mapped to close-to-zero values)

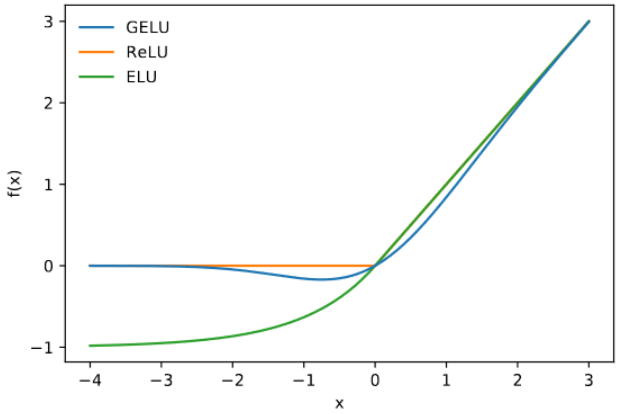
SoftPlus

- $Output = \frac{1}{\beta} \cdot \log\left(1 + e^{(\beta \cdot x)}\right)$
- Smooth approximation of ReLU
- Output always positive
- Numerical stability: use linear if $(\beta \cdot x) > threshold$



Activation Functions – Common Functions: Other ReLU variations

- Many other variations on ReLU that may include **scaling** or **learnable parameters:**
  - **GELU >> PreLU ~ ELU > SELU > CELU** (where ">" denotes how often they are used in practice)
  - GELU (or its variant GeGLU)
    - GELU = Gaussian Error Linear Unit = $x \cdot \phi(x)$, where $\phi(x)$ is the CDF of a standard normal distribution (0 mean, unit variance), and $\cdot$ is pointwise multiplication (hence gating)
    - GeGLU = Gated Linear Unit with Gaussian Error Linear Activation
    - *Standard* activations in Transformers (GELU - ViT, GeGLU - LLMs)
  - ELU (Exponential Linear Unit)
    - Used more often in Graph Neural Network codebases
    - Common in GAT (Graph Attention Networks)
    - ELU(x) = x if $x > 0$, otherwise $\alpha(e^x - 1)$; $\alpha \rightarrow$ hyperparam.

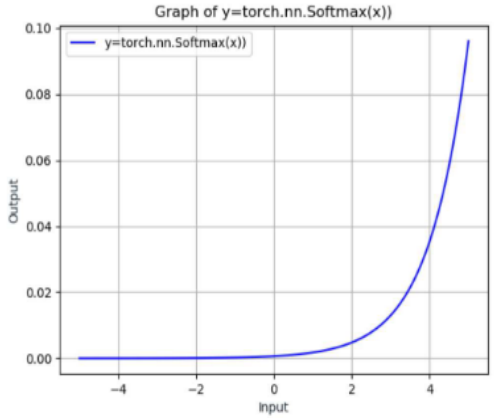


Activation Functions – Common Functions: Softmax

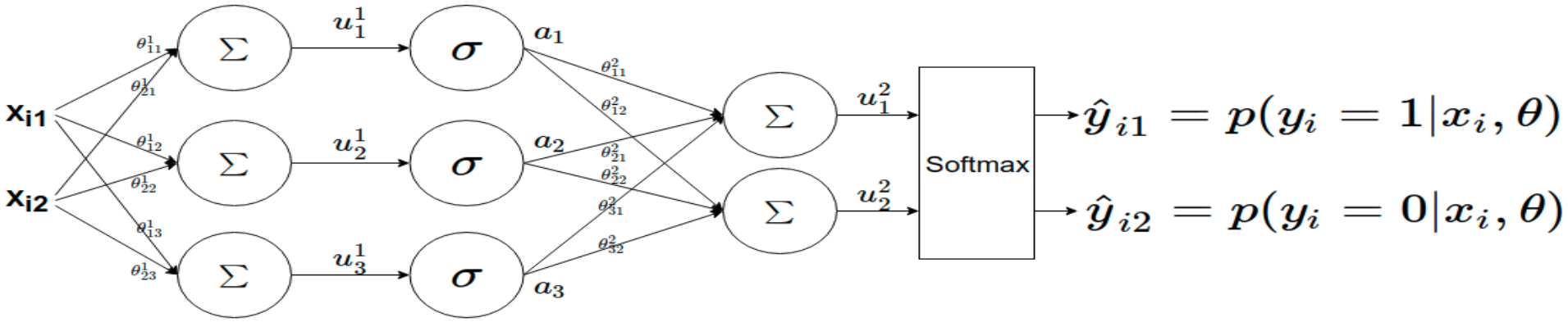
- Used usually in final network output to convert network **logits** to **class probability scores**

Softmax

- $Output = \frac{e^{x_i}}{\sum_j e^{x_j}}$
- Applies the Softmax function to an n-dimensional input Tensor rescaling them so that the elements of the n-dimensional output Tensor lie in the range [0,1] and sum up to 1.



Multi-Layer Perceptron – putting it together – a toy example



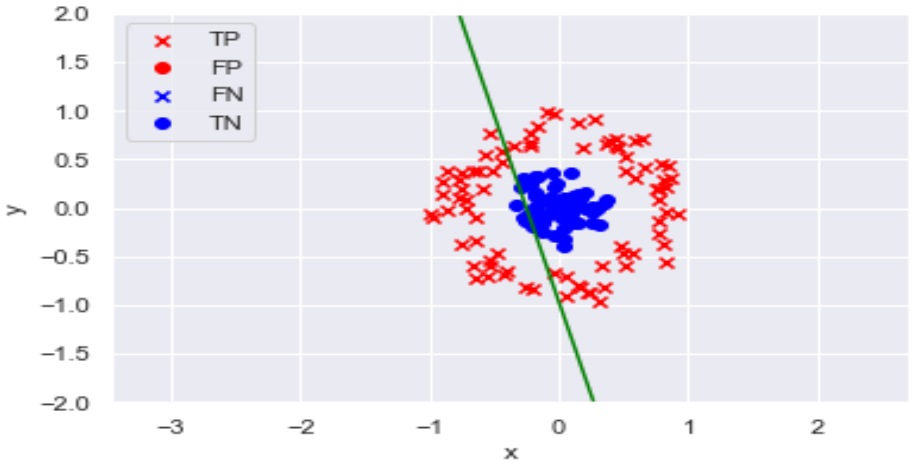
$$u_k^1 = \sum_{j=1}^{2} x_{ij}\, \theta_{jk}^1$$

$$u_k^2 = \sum_{j=1}^{3} a_j\, \theta_{jk}^2 = \sum_{j=1}^{3} \sigma(u_j^1)\, \theta_{jk}^2$$

MLP example: 2 layers with 3 and 2 neurons, respectively

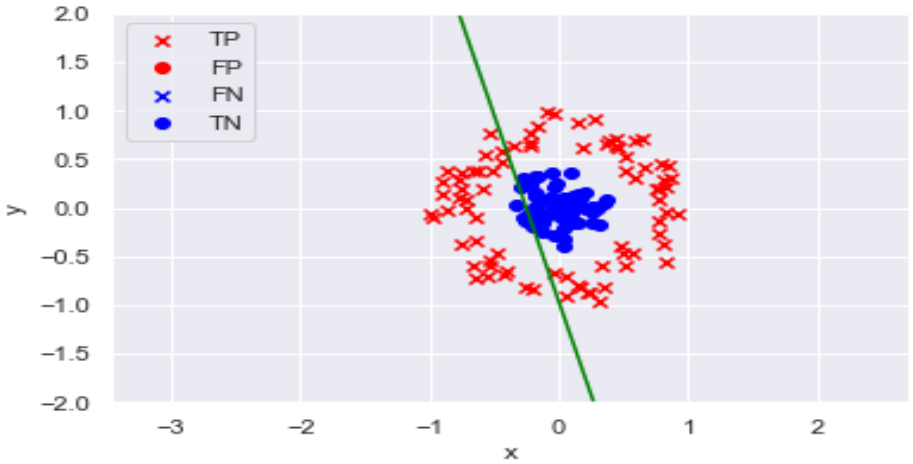
Objective: minimize *cross-entropy error*

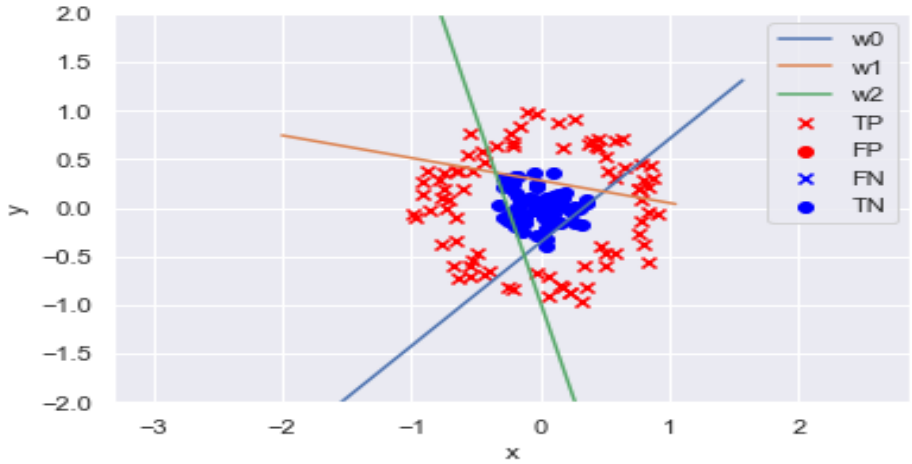
Multi-Layer Perceptron – putting it together – a toy example



Each neuron computes a *separation plane* on the space of its inputs

Multi-Layer Perceptron – putting it together – a toy example





3 input neurons create “cuts” that can isolate the inner ring

2 output neurons decide on which “side” of each cut are the positive versus the negative examples

Multi-Layer Perceptron

- Neural Net Playground (https://playground.tensorflow.org/): the simple MLP
- Observe influence of:
  - Activation Functions
  - Relation between absolute gradient value and learning rate
  - Non-normalized layer inputs
  - Batch-size

Summary

- **Gradient descent** obtains gradient values through the **Backpropagation Algorithm**
  - Computation Graph
  - Chain rule of probability
  - Gradient based update rule
- **MLP** can act as universal function approximator, but approximation depends:
  - On size of the hidden layer
  - On activation function
  - The optimizer parameters (see next Lecture)
- **Activation Functions**
  - There are some typical go-to variants: ReLU as a default, Sigmoid / Softmax for classification layer, Tanh for regression
  - One **can** obtain improvements (usually in convergence speed) if the activation function is changed to match problem characteristics
  - Many model architectures (in code bases) already have specific activation function; use it as-is (it has been most likely proven empirically to work best among alternatives)
  - The **representative power** of you training set **will usually outweigh** the contribution of a **smartly chosen activation function**